

Práctica de Laboratorio #02

Despliegue de una aplicación MEAN en el
IaaS de la ULL

Adrián Padrón Espinosa (alu0101323626@ull.edu.es)



Índice

1.	Configurar la MV que hace la función de Proxy HTTP	3
	Paso 1: Actualización del sistema e instalación de paquetes	3
	Paso 2: Configuración de Nginx como Proxy Inverso	4
	Paso 3: Verificación de sintaxis y reinicio del servicio	4
	Paso 4: Comprobación del funcionamiento de Nginx	5
2.	Configuración del servidor backend NodeJS	5
	Paso 1: Acceso temporal a internet e instalación de dependencias	5
	Paso 2: Descarga y configuración de la aplicación	6
	Paso 3: Ejecución como servicio systemd	7
	Paso 4: Verificación final desde el Proxy	7
3.	Configuración del servidor de base de datos con MongoDB	8
	Paso 1: Acceso temporal a internet	8
	Paso 2: Instalación de MongoDB 8.0	8
	Paso 3: Creación del usuario administrador	9
	Paso 4: Habilitación de la autenticación y configuración de red	9
	Paso 5: Base de datos y usuario de la aplicación	10
4.	Configuración del servidor de base de datos como sistema de ficheros compartido	11
	Paso 1: Servidor NFS en la máquina de Base de Datos	11
	Paso 2: Configuración del modo "NFSv4 only"	11
	Paso 3: Cliente NFS en la máquina Backend	12
	Paso 4: Montaje automático al arrancar	12
5.	Limpieza de interfaces temporales y aislamiento de red	13
	Paso 1: En la máquina virtual Server (Backend)	13
	Paso 2: En la máquina virtual de Base de Datos	14



1. Configurar la MV que hace la función de Proxy HTTP

En esta primera parte, procederemos a instalar y configurar el servidor web Nginx en la máquina correspondiente para que actúe como proxy inverso.

Paso 1: Actualización del sistema e instalación de paquetes

Antes de instalar nuevos servicios, actualizamos la lista de paquetes disponibles en el sistema utilizando el comando `sudo apt update`.

```
usuario@debian:~$ sudo apt update
[sudo] contraseña para usuario:
Obj:1 http://security.debian.org/debian-security bookworm-security InRelease
Obj:2 http://deb.debian.org/debian bookworm InRelease
Obj:3 http://deb.debian.org/debian bookworm-updates InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 182 paquetes. Ejecute «apt list --upgradable» para verlos.
usuario@debian:~$
```

A continuación, procedemos a instalar el servidor web ejecutando el comando `sudo apt install nginx`. Durante este proceso, el gestor de paquetes resuelve las dependencias necesarias, descarga los archivos y crea automáticamente los enlaces simbólicos para que el servicio arranque junto con el sistema.

```
usuario@debian:~$ sudo apt install nginx
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  nginx-common
Paquetes sugeridos:
  fcgiwrap nginx-doc ssl-cert
Se instalarán los siguientes paquetes NUEVOS:
  nginx nginx-common
0 actualizados, 2 nuevos se instalarán, 0 para eliminar y 182 no actualizados.
Se necesita descargar 646 kB de archivos.
Se utilizarán 1.707 kB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] S
Des:1 http://security.debian.org/debian-security bookworm-security/main amd64 nginx-common all 1.22.1-9+deb12u10 [115 kB]
Des:2 http://security.debian.org/debian-security bookworm-security/main amd64 nginx amd64 1.22.1-9+deb12u10 [531 kB]
Descargados 646 kB en 0s (2.406 kB/s)
Preconfigurando paquetes ...
Seleccionando el paquete nginx-common previamente no seleccionado.
(Leyendo la base de datos ... 45486 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../nginx-common_1.22.1-9+deb12u10_all.deb ...
Desempaquetando nginx-common (1.22.1-9+deb12u10) ...
Seleccionando el paquete nginx previamente no seleccionado.
Preparando para desempaquetar .../nginx_1.22.1-9+deb12u10_amd64.deb ...
Desempaquetando nginx (1.22.1-9+deb12u10) ...
Configurando nginx-common (1.22.1-9+deb12u10) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /lib/systemd/system/nginx.service.
Configurando nginx (1.22.1-9+deb12u10) ...
Upgrading binary: nginx.
Procesando disparadores para man-db (2.11.2-2) ...
usuario@debian:~$
```



Paso 2: Configuración de Nginx como Proxy Inverso

El siguiente paso consiste en modificar la configuración de Nginx para que redirija el tráfico entrante hacia nuestro servidor de backend. Para ello, utilizamos el editor de texto nano con privilegios de administrador para abrir el archivo de configuración por defecto ejecutando:

```
usuario@debian:~$ sudo nano /etc/nginx/sites-available/default
```

Dentro de este archivo, editamos el bloque server que escucha en el puerto 80. Específicamente, configuramos la directiva location / para establecer el proxy inverso con los siguientes parámetros:

- proxy_pass http://172.16.156.1:3000 -> Redirige todas las peticiones entrantes a la dirección IP de nuestra máquina backend (172.16.156.1) por el puerto 3000.
- proxy_http_version 1.1 -> Define la versión del protocolo HTTP.
- Cabeceras proxy_set_header -> Configuramos cabeceras adicionales como Host, X-Real-IP, X-Forwarded-For y X-Forwarded-Proto para transferir correctamente la información de la IP y la petición original del cliente hacia el backend.

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://172.16.156.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Paso 3: Verificación de sintaxis y reinicio del servicio

Tras guardar los cambios en el archivo de configuración, comprobamos que no hemos cometido errores. Ejecutamos el comando sudo nginx -t y la salida nos confirma que la sintaxis es correcta (syntax is ok) y que la prueba ha sido exitosa (test is successful).

Finalmente, aplicamos la nueva configuración reiniciando el servicio de Nginx mediante el comando sudo systemctl restart nginx.

```
usuario@debian:~$ sudo nginx -t
2026/10/07 12:32:29 [warn] 13170#13170: conflicting server name "_" on 0.0.0.0:80, ignored
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
usuario@debian:~$ sudo systemctl restart nginx
usuario@debian:~$
```



Paso 4: Comprobación del funcionamiento de Nginx

Una vez finalizada la instalación, verificamos que el servidor web está levantado y respondiendo peticiones. Para ello, lanzamos una petición HTTP local a la máquina utilizando el comando.

```
usuario@debian:~$ curl -I 127.0.0.1
HTTP/1.1 200 OK
Server: nginx/1.22.1
Date: Wed, 07 Oct 2026 11:53:46 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Mon, 05 Oct 2026 17:14:56 GMT
Connection: keep-alive
ETag: "6ac3db10-267"
Accept-Ranges: bytes
```

Esta captura confirma que recibimos un código de estado HTTP/1.1 200 OK por parte del servidor nginx/1.22.1, lo que indica que el servicio está funcionando correctamente.

2. Configuración del servidor backend NodeJS

En este apartado se despliega la aplicación express-start en la máquina backend. Se configurará para que escuche en el puerto 3000 y se ejecutará como un servicio del sistema.

Paso 1: Acceso temporal a internet e instalación de dependencias

Dado que la máquina virtual del backend se encuentra en una red interna sin salida directa a internet, es necesario habilitar temporalmente una interfaz conectada a la red externa (en este caso, ens9). Se levanta la interfaz y se solicita una dirección IP mediante DHCP:

```
usuario@debian:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:1a:4a:97:5f:25 brd ff:ff:ff:ff:ff:ff
    altname enp0s7
    inet 172.16.156.2/24 brd 172.16.156.255 scope global ens7
        valid_lft forever preferred_lft forever
    inet6 fe80::21a:4aff:fe97:5f25/64 scope link
        valid_lft forever preferred_lft forever
3: ens8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:1a:4a:97:7a:2b brd ff:ff:ff:ff:ff:ff
    altname enp0s8
    inet 172.16.157.1/24 brd 172.16.157.255 scope global ens8
        valid_lft forever preferred_lft forever
    inet6 fe80::21a:4aff:fe97:7a2b/64 scope link
        valid_lft forever preferred_lft forever
4: ens9: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 00:1a:4a:97:7a:c6 brd ff:ff:ff:ff:ff:ff
    altname enp0s9
usuario@debian:~$ sudo ip link set ens9 up
[sudo] contraseña para usuario:
usuario@debian:~$ sudo dhclient ens9
usuario@debian:~$ _
```



A continuación, actualizamos los repositorios e instalamos herramientas básicas (curl y git). Además, añadimos el repositorio oficial de NodeSource para instalar la versión 20 LTS de NodeJS, ya que la incluida por defecto en los repositorios de Debian está desactualizada.

```
usuario@debian:~$ sudo apt update
```

```
usuario@debian:~$ sudo apt install curl git -y
```

```
usuario@debian:~$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
usuario@debian:~$ sudo apt install nodejs -y
```

Tras ejecutar los comandos de instalación, verificamos la versión de Node instalada, obteniendo la v20.20.2.

Paso 2: Descarga y configuración de la aplicación

Utilizando git, clonamos el repositorio de la aplicación proporcionado para la práctica. El código fuente principal y el archivo package.json se encuentran en el subdirectorio hello, por lo que accedemos a él y ejecutamos npm install para descargar los módulos necesarios.

```
usuario@debian:~$ git clone https://github.com/crguezl/express-start.git
Clonando en 'express-start'...
remote: Enumerating objects: 142, done.
remote: Total 142 (delta 0), reused 0 (delta 0), pack-reused 142 (from 1)
Recibiendo objetos: 100% (142/142), 84.45 KiB | 1.30 MiB/s, listo.
Resolviendo deltas: 100% (69/69), listo.
usuario@debian:~$ cd express-start/hello
usuario@debian:~/express-start/hello$ npm install

added 70 packages, and audited 71 packages in 5s

28 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.8.2 -> 12.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
npm notice To update run: npm install -g npm@12.2.0
npm notice
usuario@debian:~/express-start/hello$
```

Por defecto, el código de la aplicación (hello.js) está configurado para arrancar en el puerto 8080. Como nuestro Proxy Nginx está configurado para reenviar el tráfico al puerto 3000, es necesario editar el archivo original. Modificamos la función app.listen para que escuche en el puerto 3000 y en todas las interfaces ("0.0.0.0"), permitiendo así recibir las peticiones de la red interna.

```
usuario@debian:~/express-start/hello$ nano hello.js
```

```
var server = app.listen(3000, "0.0.0.0", function () {
```



Paso 3: Ejecución como servicio systemd

Para garantizar la disponibilidad de la aplicación independientemente de las sesiones de terminal y asegurar su arranque automático tras reiniciar la máquina, creamos un archivo de servicio en `/etc/systemd/system/express-start.service`.

```
usuario@debian:~/express-start/hello$ sudo nano /etc/systemd/system/express-start.service
```

```
Unit]
Description=Aplicacion express-start (SYTW)
After=network.target

[Service]
Type=simple
User=usuario
WorkingDirectory=/home/usuario/express-start/hello
ExecStart=/usr/bin/node hello.js
Restart=on-failure

[Install]
WantedBy=multi-user.target_
```

Una vez definido el servicio con sus correspondientes rutas y usuario, recargamos los daemon de systemd, habilitamos el servicio para que inicie con el sistema y lo arrancamos. Al verificar su estado, confirmamos que se encuentra activo (running) y escuchando peticiones en el puerto 3000.

```
usuario@debian:~/express-start/hello$ sudo systemctl daemon-reload
usuario@debian:~/express-start/hello$ sudo systemctl enable --now express-start
Created symlink /etc/systemd/system/multi-user.target.wants/express-start.service → /etc/systemd/system/express-start.service.
usuario@debian:~/express-start/hello$ systemctl status express-start
● express-start.service
   Loaded: loaded (/etc/systemd/system/express-start.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-10-07 18:38:46 WEST; 4s ago
     Main PID: 7873 (node)
        Tasks: 7 (limit: 9469)
      Memory: 15.2M
         CPU: 141ms
    CGroup: /system.slice/express-start.service
            └─7873 /usr/bin/node hello.js

oct 07 18:38:47 debian node[7873]: Example app listening at http://0.0.0.0:3000
```

Paso 4: Verificación final desde el Proxy

Para comprobar la correcta comunicación entre ambas máquinas, volvemos a la máquina Proxy y realizamos una petición local con el comando `curl -I 127.0.0.1`. El servidor responde con un código 200 OK e incluye las cabeceras correspondientes, evidenciando que el tráfico pasa por Nginx y es resuelto correctamente por nuestra aplicación backend en NodeJS.

```
usuario@debian:~$ curl -I 127.0.0.1
HTTP/1.1 200 OK
Server: nginx/1.22.1
Date: Wed, 07 Oct 2026 18:18:15 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 290
Connection: keep-alive
X-Powered-By: Express
ETag: W/"122-5pVJ9hzsoA7T38HRI4XtbspVDGc"

usuario@debian:~$
```



3. Configuración del servidor de base de datos con MongoDB

En la máquina virtual de base de datos se instala MongoDB 8.0, configurando la autenticación requerida para la conexión del backend y gestionando su ejecución automática mediante systemctl.

Paso 1: Acceso temporal a internet

Se habilita temporalmente la interfaz conectada a la red externa (ens8) y se solicita una dirección IP mediante DHCP para poder descargar los paquetes.

```
usuario@debian:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:1a:4a:97:51:41 brd ff:ff:ff:ff:ff:ff
    altname enp0s7
    inet 172.16.157.2/24 brd 172.16.157.255 scope global ens7
        valid_lft forever preferred_lft forever
    inet6 fe80::21a:4aff:fe97:5141/64 scope link
        valid_lft forever preferred_lft forever
3: ens8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 00:1a:4a:97:7a:c9 brd ff:ff:ff:ff:ff:ff
    altname enp0s8
usuario@debian:~$ sudo ip link set ens8 up
[sudo] contraseña para usuario:
usuario@debian:~$ sudo dhclient ens8
usuario@debian:~$ _
```

Paso 2: Instalación de MongoDB 8.0

Se instalan las herramientas necesarias (curl y gnupg) y se añade la clave GPG junto con el repositorio oficial de MongoDB para Debian 12.

```
usuario@debian:~$ sudo apt install curl gnupg -y
```

```
usuario@debian:~$ curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
usuario@debian:~$ echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] http://repo.mongodb.org/apt/debian bookworm/mongodb-org/8.0 main" | sudo
tee /etc/apt/sources.list.d/mongodb-org-8.0.list
deb [ signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] http://repo.mongodb.org/apt/debian bookworm/mongodb-org/8.0 main
usuario@debian:~$ sudo apt update
Obj:1 http://deb.debian.org/debian bookworm InRelease
Obj:2 http://security.debian.org/debian-security bookworm-security InRelease
Obj:3 http://deb.debian.org/debian bookworm-updates InRelease
Des:4 http://repo.mongodb.org/apt/debian bookworm/mongodb-org/8.0 InRelease [2,906 B]
Des:5 http://repo.mongodb.org/apt/debian bookworm/mongodb-org/8.0/main amd64 Packages [92,6 kB]
Descargados 95,6 kB en 1s (94,1 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 170 paquetes. Ejecute «apt list --upgradable» para verlos.
```

A continuación, se instala el paquete mongodb-org en su versión 8.0, se habilita el demonio en systemctl para su inicio automático y se verifica la versión instalada (2.13.0 para mongosh).

```
usuario@debian:~$ sudo apt install mongodb-org -y
```

```
usuario@debian:~$ sudo systemctl enable --now mongod
Created symlink /etc/systemd/system/multi-user.target.wants/mongod.service → /lib/systemd/system/mongod.service.
usuario@debian:~$ mongosh --version
2.13.0
usuario@debian:~$ _
```



Paso 3: Creación del usuario administrador

Antes de activar la seguridad, se accede a la consola de MongoDB y se crea un usuario administrador global en la base de datos admin. Si se omitiera este paso, el servidor quedaría inaccesible al requerir autenticación sin existir usuarios.

```
usuario@debian:~$ mongosh
Current Mongosh Log ID: 6ac75c0863b561aa995e1113
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.13.0
Using MongoDB:  8.0.32
Using Mongosh:  2.13.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-10-08T10:00:42.384+01:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/pro
dnotes-filesystem
  2026-10-08T10:00:43.346+01:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2026-10-08T10:00:43.346+01:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
  2026-10-08T10:00:43.346+01:00: We suggest setting the contents of sysfsFile to 0.
  2026-10-08T10:00:43.346+01:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----
test> db.createUser({ user: "admin", pwd: passwordPrompt(), roles: [ { role: "UserAdminAnyDatabase", db: "admin" }, { role: "readWriteAnyDatabase", db: "admin" } ] })
Enter password
*****{ ok: 1 }
test> exit
usuario@debian:~$
```

Paso 4: Habilitación de la autenticación y configuración de red

Se edita el archivo de configuración /etc/mongod.conf.

```
usuario@debian:~$ sudo nano /etc/mongod.conf
```

En el bloque net, se modifica la directiva bindIp para que el servicio escuche tanto localmente como en la IP de la red interna de la base de datos (172.16.157.2). En el bloque security, se activa la autenticación definiendo authorization: enabled.

```
# network interfaces
net:
  port: 27017
  bindIp: 127.0.0.1,172.16.157.2,
```

```
security:
  authorization: enabled,
```

Se reinicia el servicio mongod para aplicar los cambios y se verifica mediante systemctl status que se encuentra active (running) tras las modificaciones.

```
usuario@debian:~$ sudo systemctl restart mongod
usuario@debian:~$ systemctl status mongod
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 10:08:34 WEST; 3s ago
     Docs: https://docs.mongodb.org/manual
    Main PID: 10648 (mongod)
      Memory: 199.6M
         CPU: 1.607s
    CGroup: /system.slice/mongod.service
            └─10648 /usr/bin/mongod --config /etc/mongod.conf

usuario@debian:~$
```



Paso 5: Base de datos y usuario de la aplicación

Se accede de nuevo a la consola de Mongo, esta vez forzando la autenticación como el administrador previamente creado. Se cambia a la base de datos expressdb, se crea el usuario expressuser con permisos de lectura y escritura (readWrite), y se inserta un documento de prueba para que la base de datos se genere físicamente en el almacenamiento.

```
usuario@debian:~$ mongosh -u admin -p --authenticationDatabase admin
Enter password: *****
Current Mongosh Log ID: 6ac75fa76f51364df887029d
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.13.0
Using MongoDB:      8.0.32
Using Mongosh:       2.13.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

test> use expressdb
switched to db expressdb
expressdb> db.createUser({ user: "expressuser", pwd: passwordPrompt(), roles: [ { role: "readWrite", db: "expressdb" } ] })
Enter password
*****{ ok: 1 }
expressdb> db.prueba.insertOne({ mensaje: "BD de la aplicacion creada", fecha: new Date() })
{ acknowledged: true, insertedId: ObjectId('6ac75fce6f51364df887029e') }
expressdb> _
```



4. Configuración del servidor de base de datos como sistema de ficheros compartido

En esta última fase, se configura la máquina virtual de Base de Datos para que actúe como servidor NFS exportando un directorio específico. Este directorio será montado automáticamente por la máquina virtual Backend (cliente NFS) al arrancar, restringiendo las comunicaciones estrictamente a la versión NFSv4.

Paso 1: Servidor NFS en la máquina de Base de Datos

Se instala el paquete `nfs-kernel-server`, se crea el directorio compartido `/srv/nfs/compartido` y se le asigna la propiedad al usuario del sistema.

```
usuario@debian:~$ sudo apt install nfs-kernel-server -y
```

```
usuario@debian:~$ sudo mkdir -p /srv/nfs/compartido
usuario@debian:~$ sudo chown usuario:usuario /srv/nfs/compartido
```

Para permitir el acceso exclusivo al Backend, se modifica el fichero `/etc/exports`, añadiendo la IP interna de conexión con los permisos `rw` (lectura y escritura), `sync` (escritura síncrona en disco), `no_subtree_check` y `root_squash`.

```
/srv/nfs/compartido 172.16.156.2(rw,sync,no_subtree_check,root_squash)_
```

Paso 2: Configuración del modo "NFSv4 only"

Se edita el archivo de configuración global `/etc/nfs.conf`. En la sección `[nfsd]`, se desactiva explícitamente la versión 3 (`vers3=n`) y se habilitan las versiones 4.

```
[nfsd]
# debug=0
# threads=8
# host=
# port=0
# grace-time=90
# lease-time=90
# udp=n
# tcp=y
vers3=n_
vers4=y
# vers4.0=y
vers4.1=y
vers4.2=y
# rdma=n
# rdma-port=20049
```



Dado que NFSv4 no requiere de los servicios antiguos rpcbind y rpc-statd, estos se desactivan y enmascaran mediante systemctl. Finalmente, se reinicia y habilita el servicio nfs-server, verificando en /proc/fs/nfsd/versions que solo las versiones 4 se encuentran activas (-2 -3 +4 +4.1 +4.2).

```
usuario@debian:~$ sudo systemctl mask --now rpc-statd.service rpcbind.service rpcbind.socket
usuario@debian:~$ sudo systemctl restart nfs-server
usuario@debian:~$ sudo systemctl enable nfs-server
usuario@debian:~$ sudo cat /proc/fs/nfsd/versions
-2 -3 +4 +4.1 +4.2
usuario@debian:~$
```

Paso 3: Cliente NFS en la máquina Backend

En la máquina virtual Server (Backend), se instalan las utilidades de cliente mediante el paquete nfs-common y se preparan los directorios necesarios para el montaje de red.

```
usuario@debian:~$ sudo apt install nfs-common -y
```

```
usuario@debian:~$ sudo apt install nfs-common -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
nfs-common ya está en su versión más reciente (1:2.6.2-4+deb12u1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 169 no actualizados.
usuario@debian:~$ sudo mkdir -p /mnt/compartido
usuario@debian:~$ sudo mount -t nfs4 172.16.157.2:/srv/nfs/compartido /mnt/compartido
usuario@debian:~$ touch /mnt/compartido/prueba_desde_backend.txt
```

Paso 4: Montaje automático al arrancar

Para garantizar la persistencia tras un reinicio, se configura el montaje automático añadiendo la entrada correspondiente en /etc/fstab. Se incluye la directiva _netdev para indicar al sistema que espere a que la red esté operativa antes de intentar montar el recurso remoto.

```
172.16.157.2:/srv/nfs/compartido /mnt/compartido nfs4 defaults,_netdev 0 0_
```

Tras aplicar los cambios con mount -a, se verifica mediante df -h que el sistema de ficheros de red se encuentra correctamente montado en /mnt/compartido y plenamente accesible.

```
usuario@debian:~$ sudo mount -a
usuario@debian:~$ df -h | grep nfs
172.16.157.2:/srv/nfs/compartido 24G 3,2G 19G 15% /mnt/compartido
```



5. Limpieza de interfaces temporales y aislamiento de red

Para garantizar la máxima seguridad de la arquitectura de tres capas y cumplir con el diseño de red aislado, una vez finalizadas todas las instalaciones de software y configuraciones, se procede a eliminar las interfaces de red temporales que proveían acceso a internet desde el panel de virtualización.

Paso 1: En la máquina virtual Server (Backend)

Se elimina la interfaz nic3 conectada a la red externa (DOC1), la cual había obtenido dinámicamente la IP 10.6.129.92. De esta manera, el servidor de aplicaciones queda comunicado exclusivamente a través de sus dos redes internas: DOCINT1 (hacia el Proxy Inverso) y DOCINT2 (hacia la Base de Datos).

Interfaces de red 3

[+ Crear NIC](#)



nic1 (DOCINT1/DOCINT1)
IPv4: 172.16.156.2
IPv6: fe80::21a:4aff:fe97:5f25

 



nic2 (DOCINT2/DOCINT2)
IPv4: 172.16.157.1
IPv6: fe80::21a:4aff:fe97:7a2b

 



nic3 (DOC1/DOC1)
IPv4: 10.6.129.92
IPv6: fe80::21a:4aff:fe97:7ac6

 

Interfaces de red 2

[+ Crear NIC](#)



nic1 (DOCINT1/DOCINT1)
IPv4: 172.16.156.2
IPv6: fe80::21a:4aff:fe97:5f25

 



nic2 (DOCINT2/DOCINT2)
IPv4: 172.16.157.1
IPv6: fe80::21a:4aff:fe97:7a2b

 



Paso 2: En la máquina virtual de Base de Datos

Se elimina la interfaz nic2 conectada a la red externa (DOC1), que contaba con la IP 10.6.129.90. El servidor de base de datos queda así totalmente aislado del exterior y protegido, siendo accesible de forma exclusiva desde el Backend a través de la red interna DOCINT2 mediante su IP estática 172.16.157.2.

Interfaces de red 2

+ Crear NIC

↑ nic1 (DOCINT2/DOCINT2)

IPv4: 172.16.157.2
IPv6: fe80::21a:4aff:fe97:5141

↑ nic2 (DOC1/DOC1)

IPv4: 10.6.129.90
IPv6: fe80::21a:4aff:fe97:7ac9

Interfaces de red 1

+ Crear NIC

↑ nic1 (DOCINT2/DOCINT2)

IPv4: 172.16.157.2
IPv6: fe80::21a:4aff:fe97:5141